

A3 · Answer Key and Demo Script

CS425 · Name the Layer

INSTRUCTOR COPY

Graded pass/fail. A3 runs on A2's testbed and needs stations 1, 2, 3 and 7 live. Relaunch if you tore it down, and **put the new addresses on the board**: they will not match A2's, which is worth saying out loud, because it is the whole reason the worksheet is written in terms of ALPHA and BRAVO.

```
$ ./scripts/cs425/a2-connectivity-triage.sh verify # expect 9 passed, 0 failed
$ ./scripts/cs425/a2-connectivity-triage.sh card   # the two IPs, for the board
$ ALPHA=35.90.160.84
```

ROUND 1 · THE DECISION TREE

Q1. Does the name resolve, and to what address?

NXDOMAIN. cause: no such name · layer: application (DNS) · fixed by: whoever owns the zone. A2 station 4. Zero packets sent to the target.

Resolves to an unroutable address. cause: the record is wrong, RFC 1918 · layer: application, symptom at the network layer · fixed by: whoever owns the record. A2 station 5.

Q2. Does the TCP connection complete?

Instant, "refused", host sent a RST. cause: nothing listening on that port · layer: transport · fixed by: whoever runs the server. A2 station 3.

Hangs, nothing came back. cause: a packet filter dropping silently · layer: filter acting on transport ports at the network edge · fixed by: whoever runs the network or the security group. A2 station 2.

Q3. Does the application actually send a response?

Open, no bytes. cause: the server accepted and never wrote · layer: application · fixed by: whoever runs the server. Station 7, today's Round 2.

It works. If ping still fails, that tells you **only that ICMP is filtered**, and nothing about the service. A2 station 6.

The ordering is the content. Q1 has to come first because a name failure means **no packet was ever sent**, so nothing below it can possibly be the cause. Q3 has to come last because it is the only question you cannot ask until a connection exists. A group that puts "can I ping it" anywhere in this tree has missed Round 3 of A2, and that is worth catching.

PASS BAR: Q1 is about name resolution and comes first, every leaf has something in the layer and owner slots, and all six A2 stations land somewhere in the summary table.

ROUND 2 · STATION 7, THE STALL

Station 7 · the handshake that leads nowhere

```
$ nc -vz -G 5 $ALPHA 8084 # -G on macOS, -w on Linux
Connection to 35.90.160.84 port 8084 [tcp/*] succeeded!

$ curl -sS -o /dev/null -m 6 -w 'connect=%{time_connect}\n' http://$ALPHA:8084/
curl: (28) Operation timed out after 6006 milliseconds with 0 bytes received
connect=0.024711

# for contrast, station 2 from A2:
curl: (28) Connection timed out after 6003 milliseconds
connect=0.000000
```

SAY: both are exit 28, and they are different failures. Station 2 says "CONNECTION timed out" with connect=0.000000: the handshake never happened. Station 7 says "OPERATION timed out with 0 bytes received" and connect=0.024711, a real number. Transport succeeded. The application never wrote.

1. Which layer completed, which never did?

Transport completed, which is why nc -vz is happy. The application never did.

2. What does the connect= difference tell you?

It is the dividing line. A zero means the fault is at or below transport; a real value with a timeout after it means transport is fine and the fault is above it. This single field turns one symptom into two diagnoses.

3. What does the load balancer's health check say, and why is that worse than nothing?

It says **healthy**. That is worse than no check because it actively keeps a wedged instance in the pool, hides the outage behind a green dashboard, and sends real users to a server that will never answer them. A check that opens a port tests the kernel, not the service. A useful check reads a response.

If a group gets fired up about this one, they are right to: it is one of the most common real world outages there is.

4. Where does it land on the tree?

Under Q3, the branch most groups do not have until they meet this station. Adding the branch is the answer; a group that says "our tree was missing this" has done the round correctly.

PASS BAR: the two timeout messages are distinguished, and the tree has a branch for "connected and nothing came back".

ROUND 3 · PREDICT, THEN WATCH

Do not run the capture until every group has written a prediction. The point of the round is that a wrong prediction is informative, and it stops being informative once they have seen the answer.

The demo · run this on ALPHA, project it

```
$ ./scripts/cs425/a2-connectivity-triage.sh ssh -- \
  sudo tcpdump -nni any 'tcp port 8081 or tcp port 8082'
```

a volunteer probes 8081 ... nothing appears at all ...

the same volunteer probes 8082:

01:09:55.964937 ens5 In IP 135.134.183.169.64447 > 172.31.1.125.8082: Flags [S], length 0

01:09:55.964963 ens5 Out IP 172.31.1.125.8082 > 135.134.183.169.64447: Flags [R.], win 0, length 0

SAY: port 8081 produces **NO PACKETS AT ALL**. Port 8082 shows the SYN arriving and the RST leaving **26 MICROSECONDS LATER**. Same host, same capture, same instant. Use `-nn`, not `-n`, or `tcpdump` renames 8082 to a service name and the output stops being readable.

The answer: before or after?

Before. An AWS security group is enforced at the virtual network interface, ahead of delivery to the instance, so the host genuinely never sees the SYN for 8081. The absence of packets is the evidence.

The useful wrong prediction is "they arrived and something on the host rejected them". That is exactly what a *host* firewall like `iptables` would do, and the capture would then show the SYNs arriving. The client cannot tell those two apart, which is the entire reason you go and look at the server. Give full credit for it and use it at the board.

PASS BAR: a prediction was written before the demo, the command names a packet capture tool, and the group recorded that 8081 showed nothing while 8082 showed a SYN and a RST.

ROUND 4 · SAY IT OUT LOUD

1. The two symptom words, and what the server sends.

Timeout and **refused**. On a refusal the kernel sends a **TCP RST**. On a timeout the server **sends nothing at all**, which is why it takes seconds instead of milliseconds: the client is waiting out its own retransmissions rather than reacting to a reply.

2. One command, three outcomes.

The intended answer is one `curl -v --connect-timeout 5` at the real endpoint, because it walks all three layers and reports where it stopped: **(a)** could not resolve, so it is DNS and no packet was sent; **(b)** refused or timed out, separating a live host with a dead service from a path that drops; **(c)** a status line, so the service is up and the reporter's problem is elsewhere.

Any command with three genuinely distinct outcomes passes. A group that picks `dig` and argues resolution is cheapest to rule out first has made a real argument; take it. A group that picks **ping** has not, and A2 Round 3 is the counterexample they collected themselves.

PASS BAR: a command is named and three outcomes are listed. Whether the choice is the one you would make matters less than whether they can say what the output would tell them.

Running the room

- **A group without their A2 data.** Hand them the mechanism table from the A2 key rather than letting them burn the period re-probing stations 1 through 6.
- **Round 1 is the deliverable.** If the period runs short, compress Round 2 to questions 1 and 3 and keep Round 3, which needs you at the projector anyway.
- **The tree is the artifact they keep,** so say that out loud before they hand it in. The stretch, compressing it to five runbook lines with no diagram, is the real test of whether they understood it.

When the last section is done

Nothing expires on its own. `./scripts/cs425/a2-connectivity-triage.sh destroy --purge-key`